Team Delivery Package: Machine Learning Engine

Enterprise Milestone-Based Technical Specifications

MODULE NAME:	Team 5: Machine Learning Engine
GIT BRANCH:	feature/modeling
LEAD MEMBERS:	Mohamed Khaled El-Shayp, Ahmed Gamal
WORKLOAD TIME:	3 weeks
KICKOFF DATE:	11 July 2026
CODE FREEZE:	23 July 2026
VERSION:	v1.2.0

1. Cover Page & Metadata Summary

This delivery package establishes the contract specifications for the **Team 5: Machine Learning Engine**.

Milestone	Target Date	Status	Description
Project Kickoff	11 July 2026	COMPLETED	Milestone scope locked & repository structured.
Code Freeze	23 July 2026	IN PROGRESS	All functional packages and tests locked.
Integration Phase	24 July 2026	PLANNED	Multi-team pipeline joins and bug resolution.
Final Presentation	25 July 2026	PLANNED	Final delivery and platform presentation.

2. Team Overview & Assignments

Resource Details

Lead Team Members: **Mohamed Khaled El-Shayp, Ahmed Gamal**

Branch Space: **`feature/modeling`**

Role Assignment: **Machine Learning Architects**

Difficulty Rating: **High** | Priority Index: **High** | Est. Workload: **3 weeks**

3. Business & Technical Goals

Business Objective

Train baseline models automatically to predict user target variables.

Technical Objective

Train Random Forest estimators and automate task inferences (classification vs regression).

Position inside AutoAnalyst AI

Predictive model training core.

4. System Architecture & folder layout

Assigned folder Tree Structure

```
src/autoanalyst/
■■■ modeling/
    ■■■ __init__.py
    ■■■ classification.py
    ■■■ regression.py
```

Target Code Modules

Your codebase modifications belong inside: `src/autoanalyst/modeling/classification.py, src/autoanalyst/modeling/regression.py`.

Functional Code Interface Specifications

```
# src/autoanalyst/modeling/classification.py
import pandas as pd
from sklearn.ensemble import RandomForestClassifier

class ClassificationModel:
    def __init__(self, random_state: int = 42):
        self.model = RandomForestClassifier(random_state=random_state)

    def train(self, X: pd.DataFrame, y: pd.Series, test_size: float = 0.2):
        """Trains RandomForestClassifier on stratified splits."""
        pass

    def predict(self, X: pd.DataFrame):
        pass
```

API Specification

```
ClassificationModel.train(X: pd.DataFrame, y: pd.Series, test_size: float)
Returns:
- Tuple[X_train, X_test, y_train, y_test] resulting from internal data splitting.
```

5. Git Workflow & Code Integration Policy

Branching Workflow

All developers pull and write inside `feature/modeling` created from the latest `develop` branch.

Pull Request Requirements

1. Merge latest develop branch changes into features before raising PRs.
2. Assign team lead reviews and resolve conflicts.
3. A minimum of one approved review check is required.

6. Quality Standards & Guidelines

Format & Lint Rules

All variables, methods, and file layouts must use lower_case snake_case parameters. Code comments must be in English.

Error Checking & Imputers

All functional blocks should be guarded by custom exception checks (raise ValueError/KeyError).

Logging Rules

Incorporate logging warnings rather than std print statements.

7. Expected Deliverables & Verification

Required Files list

`src/autoanalyst/modeling/classification.py, src/autoanalyst/modeling/regression.py`

Verification Code Assertions

Unit test validations are situated in: `tests/test_modeling.py`.
Pipeline integration assertions belong in: `tests/test_end_to_end_pipeline.py`.

IMPORTANT GUIDELINE
All functional checks must compile successfully with zero syntax warnings before code freeze.

8. Definition of Done Checklist

- [ ] All functions compile cleanly with PEP 8 checks.
- [ ] Unit tests cover all key methods and functions.
- [ ] Try-except checks isolate external dataframe issues safely.
- [ ] The PR has been reviewed and verified by Team 1.

9. Common Mistakes & Troubleshooting

Pitfalls to Avoid

- Fitting models on dataset arrays that contain object data types or null values.
- Omitting class stratification when labels have skewed ratios, causing training bias.
- Calling train/test splits inside loops, leaking prediction data.